

2º curso / 2º cuatr.

Grado en
Ingeniería
Informática

Arquitectura de Computadores (AC)

Cuaderno de prácticas.

Bloque Práctico 1. Programación paralela I: **Directivas** OpenMP

Estudiante (nombre y apellidos): David Rodriguez Aparicio

Parte I. Ejercicios basados en los ejemplos del seminario práctico

1. Usar la directiva parallel combinada con directivas de trabajo compartido en los ejemplos bucle-for.c y sections.c del seminario. Incorporar el código fuente resultante al cuaderno de prácticas.

RESPUESTA: Captura que muestre el código fuente bucle-forModificado.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  int main(int argc, char* argv[]){
6      int i, n = 9;
7      if(argc < 2){
8          fprintf(stderr, "\n[ERROR] - Falta no iteraciones \n");
9          exit(-1);
10     }
11
12     n = atoi(argv[1]);
13
14     #pragma omp parallel for
15
16     for(i=0; i<n; i++){
17         printf("thread %d ejecuta la iteración %d del bucle\n", omp_get_thread_num(), i);
18     }
19
20     return 0;
21 }
```

RESPUESTA: Captura que muestre el código fuente sectionsModificado.c

```

sectionsModificado.c U X
AC > Practica 1 (+ S1) > Codigos > sectionsModificado.c > ...
1  #include <stdio.h>
2  #include <omp.h>
3
4  void funcA(){
5      printf("En funcA: esta sección la ejecuta el thread %d\n", omp_get_thread_num());
6  }
7
8  void funcB(){
9      printf("En funcB: esta sección la ejecuta el thread %d\n", omp_get_thread_num());
10 }
11
12 void main(){
13     #pragma omp parallel sections
14     {
15         #pragma omp section
16         funcA();
17
18         #pragma omp section
19         funcB();
20     }
21 }
22

```

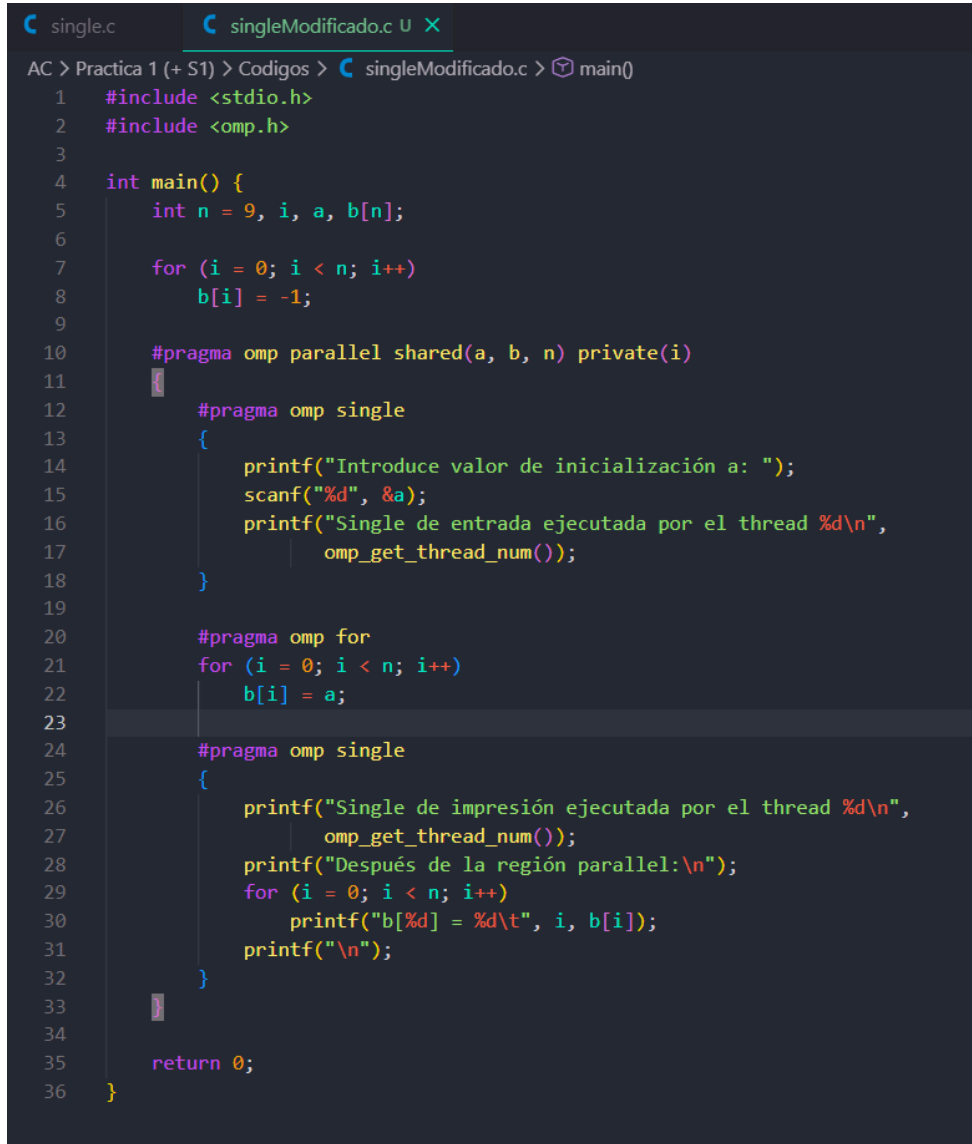
```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ export OMP_NUM_THREADS=8
./bucle-forModificado 8
./sectionsModificado
thread 1 ejecuta la iteración 1 del bucle
thread 3 ejecuta la iteración 3 del bucle
thread 0 ejecuta la iteración 0 del bucle
thread 6 ejecuta la iteración 6 del bucle
thread 5 ejecuta la iteración 5 del bucle
thread 7 ejecuta la iteración 7 del bucle
thread 4 ejecuta la iteración 4 del bucle
thread 2 ejecuta la iteración 2 del bucle
En funcA: esta sección la ejecuta el thread 1
En funcB: esta sección la ejecuta el thread 2

```

2. Imprimir los resultados del programa single.c usando una directiva single dentro de la construcción parallel en lugar de imprimirlos fuera de la región parallel. Añadir lo necesario, dentro de la nueva directiva single incorporada, para que se imprima el identificador del thread que ejecuta el bloque estructurado de la directiva single. Incorpore en su cuaderno de trabajo el código fuente y volcados de pantalla con los resultados de ejecución obtenidos.

RESPUESTA: Captura que muestre el código fuente singleModificado.c



```

single.c  singleModificado.c U X
AC > Practica 1 (+ S1) > Codigos > singleModificado.c > main()
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int n = 9, i, a, b[n];
6
7      for (i = 0; i < n; i++)
8          b[i] = -1;
9
10     #pragma omp parallel shared(a, b, n) private(i)
11     {
12         #pragma omp single
13         {
14             printf("Introduce valor de inicialización a: ");
15             scanf("%d", &a);
16             printf("Single de entrada ejecutada por el thread %d\n",
17                 omp_get_thread_num());
18         }
19
20         #pragma omp for
21         for (i = 0; i < n; i++)
22             b[i] = a;
23
24         #pragma omp single
25         {
26             printf("Single de impresión ejecutada por el thread %d\n",
27                 omp_get_thread_num());
28             printf("Después de la región parallel:\n");
29             for (i = 0; i < n; i++)
30                 printf("b[%d] = %d\t", i, b[i]);
31             printf("\n");
32         }
33     }
34
35     return 0;
36 }

```

CAPTURAS DE PANTALLA:

```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ export OMP_NUM_THREADS=4
./singleModificado
Introduce valor de inicialización a: 8
Single de entrada ejecutada por el thread 2
Single de impresión ejecutada por el thread 1
Después de la región parallel:
b[0] = 8      b[1] = 8      b[2] = 8      b[3] = 8      b[4] = 8      b[5] = 8      b[6] = 8      b[7] = 8      b[8] = 8
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ ./singleModificado
Introduce valor de inicialización a: 7+
Single de entrada ejecutada por el thread 1
Single de impresión ejecutada por el thread 0
Después de la región parallel:
b[0] = 7      b[1] = 7      b[2] = 7      b[3] = 7      b[4] = 7      b[5] = 7      b[6] = 7      b[7] = 7      b[8] = 7
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$

```

3. Imprimir los resultados del programa single.c usando una directiva master dentro de la construcción parallel en lugar de imprimirlos fuera de la región parallel. Añadir lo necesario, dentro de la nueva directiva master incorporada, para que se imprima el identificador del thread que ejecuta el bloque estructurado de la directiva master. Incorpore en su cuaderno el código fuente y volcados de pantalla con los resultados de ejecución obtenidos. ¿Qué diferencia observa con respecto a los resultados de ejecución del ejercicio anterior?

RESPUESTA: Captura que muestre el código fuente singleModificado2.c

```

C singleModificado2.c U X
AC > Practica 1 (+ S1) > Codigos > C singleModificado2.c > main()
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int n = 9, i, a, b[n];
6
7      for (i = 0; i < n; i++)
8          b[i] = -1;
9
10     #pragma omp parallel shared(a, b, n) private(i)
11     {
12         #pragma omp single
13         {
14             printf("Introduce valor de inicialización a: ");
15             scanf("%d", &a);
16             printf("Single de entrada ejecutada por el thread %d\n",
17                 omp_get_thread_num());
18         }
19
20         #pragma omp for
21         for (i = 0; i < n; i++)
22             b[i] = a;
23
24         #pragma omp barrier
25
26         #pragma omp master
27         {
28             printf("Master de impresión ejecutada por el thread %d\n",
29                 omp_get_thread_num());
30             printf("Después de la región parallel:\n");
31             for (i = 0; i < n; i++)
32                 printf("b[%d] = %d\t", i, b[i]);
33             printf("\n");
34         }
35     }
36
37     return 0;
38 }

```

CAPTURAS DE PANTALLA:

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ ./singleModificado2
Introduce valor de inicialización a: 4
Single de entrada ejecutada por el thread 0
Master de impresión ejecutada por el thread 0
Después de la región parallel:
b[0] = 4      b[1] = 4      b[2] = 4      b[3] = 4      b[4] = 4      b[5] = 4      b[6] = 4      b[7] = 4      b[8] = 4
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ ./singleModificado2
Introduce valor de inicialización a: 2
Single de entrada ejecutada por el thread 2
Master de impresión ejecutada por el thread 0
Después de la región parallel:
b[0] = 2      b[1] = 2      b[2] = 2      b[3] = 2      b[4] = 2      b[5] = 2      b[6] = 2      b[7] = 2      b[8] = 2
```

RESPUESTA A LA PREGUNTA:

Con single el bloque es ejecutado por cualquier hilo, mientras que con master siempre usa el hilo principal (thread 0), además master no tiene barrera implícita y si es necesario hay que añadirla manualmente.

4. ¿Por qué si se elimina directiva barrier en el ejemplo master.c la suma que se calcula e imprime no siempre es correcta? Responda razonadamente.

RESPUESTA: Porque no terminan de sincronizarse los hilos y el hilo master puede mostrar el resultado antes de lo previsto.

Parte II. Resto de ejercicios (usar en atcgrid la cola ac a no ser que se tenga que usar atcgrid4)

5. El programa secuencial C del Listado 1 (ver cuaderno de BP0) calcula la suma de dos vectores ($v3 = v1 + v2$; $v3(i) = v1(i) + v2(i)$, $i=0, \dots, N-1$). Generar el ejecutable del programa del Listado 1 para **vectores globales**. Usar time (Lección 3/ Tema 1) en la línea de comandos para obtener, en atcgrid, el tiempo de ejecución (*elapsed time*) y el tiempo de CPU del usuario y del sistema generado. Obtenga los tiempos para vectores con 10000000 componentes. ¿La suma de los tiempos de CPU del usuario y del sistema es menor, mayor o igual que el tiempo real (*elapsed*)? Justifique la respuesta.

CAPTURAS DE PANTALLA:

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ time ./SumaVectoresC-global 10000000
Tiempo(seg.):0.101053265      / Tamaño Vectores:10000000      / V1[0]+V2[0]=V3[0](0.905589+0.631330=1.536919) // V1[9999999]+V2[9999999]=V3[9999999](0.601285+0.888755=1.490040) /

real    0m0.377s
user    0m0.179s
sys     0m0.199s
```

RESPUESTA: Es casi igual, porque el programa es secuencial y usa únicamente un hilo.

6. Generar el código ensamblador a partir del programa secuencial C del Listado 1 (ver cuaderno de BP0) para **vectores globales** (para generar el código ensamblador tiene que compilar usando -S en lugar de -o). Utilice el fichero con el código fuente ensamblador generado y el fichero ejecutable generado en el ejercicio 5 para obtener para atcgrid los MIPS (*Millions of Instructions Per Second*) y los MFLOPS (*Millions of Floating-point Per Second*) del código que obtiene la suma de vectores (código entre las funciones clock_gettime()); el cálculo se debe hacer para 10 y 10000000 componentes en los vectores (consulte la Lección 3/Tema1 AC). Razonar cómo se han obtenido los valores que se necesitan para calcular los MIPS y MFLOPS. Incorporar **el código ensamblador de la parte de la suma de vectores** (no de todo el programa) en el cuaderno.

CAPTURAS DE PANTALLA (que muestren la generación del código ensamblador y del código ejecutable, y la obtención de los tiempos de ejecución):

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ ./SumaVectoresC-global 10
Tiempo(seg.):0.000000100 / Tamaño Vectores:10 / V1[0]+V2[0]=V3[0](0.586146+0.824673=1.410819) // V1[9]+V2[9]=V3[9](0.111804+0.485985=0.597789) /
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ ./SumaVectoresC-global 10000000
Tiempo(seg.):0.091126568 / Tamaño Vectores:10000000 / V1[0]+V2[0]=V3[0](0.069355+0.643035=0.712390) // V1[9999999]+V2[9999999]=V3[9999999](0.832136+0.913177=1.745313) /

david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ gcc -O2 SumaVectoresC-global.c -o SumaVectoresC-global -lrt^C
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ gcc -O2 -S SumaVectoresC-global.c -o SumaVectoresC-global.s -lrt^C
```

RESPUESTA: cálculo de los MIPS y los MFLOPS

N es el numero de instrucciones

$MFLOPS = (N \times 1) / (\text{tiempo} \times 10^6)$

$MIPS = (N \times 6) / (\text{tiempo} \times 10^6)$

(N = 10)

$MFLOPS = 10 / (0.000000100 \times 10^6) = 100$

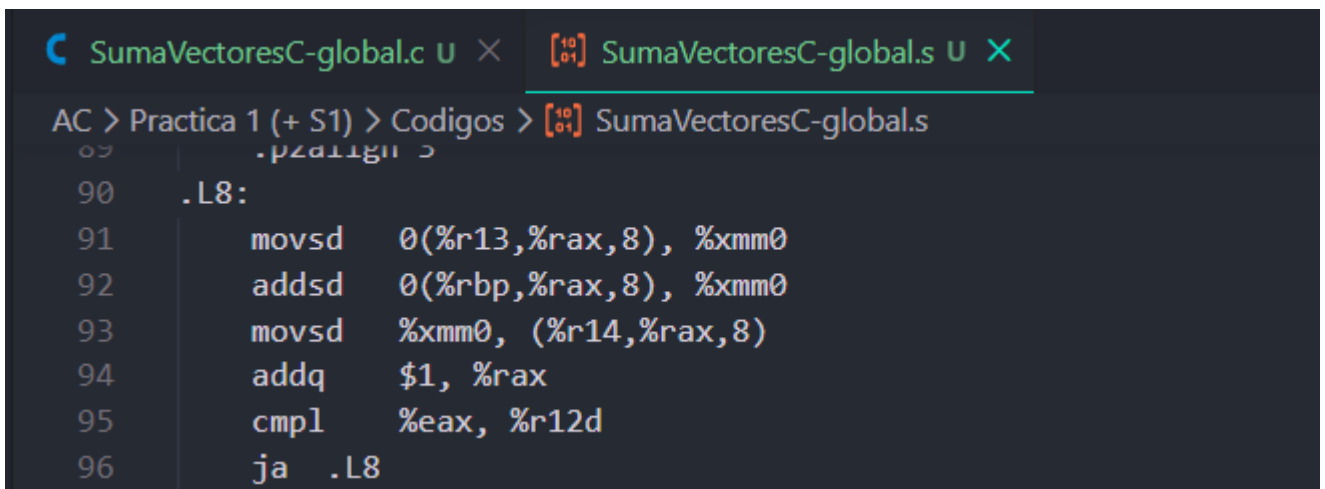
$MIPS = (10 \times 6) / (0.000000100 \times 10^6) = 600$

(N = 10000000)

$MFLOPS = 10000000 / (0.091126568 \times 10^6) = 109.7375$

$MIPS = (10000000 \times 6) / (0.091126568 \times 10^6) = 658.4249$

RESPUESTA: Captura que muestre el código ensamblador generado de la parte de la suma de vectores



```
SumaVectoresC-global.c U X SumaVectoresC-global.s U X
AC > Practica 1 (+ S1) > Codigos > SumaVectoresC-global.s
09      .p2align 5
90      .L8:
91      movsd    0(%r13,%rax,8), %xmm0
92      addsd    0(%rbp,%rax,8), %xmm0
93      movsd    %xmm0, (%r14,%rax,8)
94      addq     $1, %rax
95      cmpl     %eax, %r12d
96      ja      .L8
```


7. Implementar un programa en C con OpenMP, a partir del código del Listado 1, que calcule en paralelo la suma de dos vectores ($v3 = v1 + v2$; $v3(i)=v1(i)+v2(i)$, $i=0,\dots,N-1$) usando las directivas `parallel` y `for`. Se debe paralelizar también las tareas asociadas a la inicialización de los vectores. Para dicha inicialización, escoja el modo de inicialización de vectores pequeños en lugar del que usa aleatorios. ¿Qué problema podría haber si se usara `drand_48` (Pista: *thread safety*)? Como en el código del Listado 1 se debe obtener el tiempo (*elapsed time*) que supone el cálculo de la suma. Para obtener este tiempo usar la función `omp_get_wtime()`, que proporciona el estándar OpenMP, en lugar de `clock_gettime()`.

NOTAS: (1) el número de componentes N de los vectores debe ser un argumento de entrada al programa; (2) se deben inicializar los vectores antes del cálculo; (3) se debe asegurar que el programa calcula la suma correctamente imprimiendo todos los componentes del vector resultante, $v3$, para varios tamaños pequeños de los vectores (por ejemplo, $N = 8$ y $N=11$); (4) se debe imprimir el tamaño de los vectores y el número de hilos; (5) se debe imprimir sea cual sea el tamaño de los vectores el tiempo de ejecución del código paralelo que suma los vectores y, al menos, el primer y último componente de $v1$, $v2$ y $v3$ (esto último evita que las optimizaciones del compilador eliminen el código de la suma).

Con `drand48()` podría haber problemas ya que no es seguro, por lo que varios hilos podrían acceder a la vez generando problemas

RESPUESTA: Captura que muestre el código fuente implementado `sp-OpenMP-for.c`

```

C sumaVectorOMP.c U
AC > Practica 1 (+ S1) > Codigos > sumaVectorOMP.c > main(int, char **)
11  int main(int argc, char **argv) {
12      int i;
13      unsigned int N;
14      double t1, t2, tiempo;
15      int nthreads = 0;
16
17      if (argc < 2) {
18          printf("Faltan nº componentes del vector\n");
19          exit(-1);
20      }
21
22      N = atoi(argv[1]);
23
24      #ifdef VECTOR_GLOBAL
25      if (N > MAX) N = MAX;
26      #endif
27
28      #pragma omp parallel
29      {
30          #pragma omp single
31          {
32              nthreads = omp_get_num_threads();
33          }
34
35          #pragma omp for
36          for (i = 0; i < (int)N; i++) {
37              v1[i] = N * 0.1 + i * 0.1;
38              v2[i] = N * 0.1 - i * 0.1;
39          }
40      }
41
42
43      t1 = omp_get_wtime();
44
45      #pragma omp parallel for
46      for (i = 0; i < (int)N; i++) {
47          v3[i] = v1[i] + v2[i];
48      }
49
50      t2 = omp_get_wtime();
51      tiempo = t2 - t1;
52
53      printf("Tamaño vectores: %u\n", N);
54      printf("Numero de hilos: %d\n", nthreads);
55      printf("Tiempo(seg.): %11.9f\n", tiempo);
56
57      if (N > 0) {
58          printf("V1[0]=%8.6f V2[0]=%8.6f V3[0]=%8.6f\n",
59              v1[0], v2[0], v3[0]);
60          printf("V1[%u]=%8.6f V2[%u]=%8.6f V3[%u]=%8.6f\n",
61              N-1, v1[N-1], N-1, v2[N-1], N-1, v3[N-1]);
62      }
63
64      if (N < 12) {
65          printf("Vector resultado completo:\n");
66          for (i = 0; i < (int)N; i++) {
67              printf("V1[%d]+V2[%d]=V3[%d](%8.6f+%8.6f=%8.6f)\n",
68                  i, i, i, v1[i], v2[i], v3[i]);
69          }
70      }

```

(RECUERDE ADJUNTAR CÓDIGO FUENTE AL .ZIP)

CAPTURAS DE PANTALLA (compilación y ejecución para N=8 y N=11):

```
david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ export OMP_NUM_THREADS=4
./sumaVectorOMP 8
./sumaVectorOMP 11
Tamaño vectores: 8
Numero de hilos: 4
Tiempo(seg.): 0.000002231
V1[0]=0.800000 V2[0]=0.800000 V3[0]=1.600000
V1[7]=1.500000 V2[7]=0.100000 V3[7]=1.600000
Vector resultado completo:
V1[0]+V2[0]=V3[0](0.800000+0.800000=1.600000)
V1[1]+V2[1]=V3[1](0.900000+0.700000=1.600000)
V1[2]+V2[2]=V3[2](1.000000+0.600000=1.600000)
V1[3]+V2[3]=V3[3](1.100000+0.500000=1.600000)
V1[4]+V2[4]=V3[4](1.200000+0.400000=1.600000)
V1[5]+V2[5]=V3[5](1.300000+0.300000=1.600000)
V1[6]+V2[6]=V3[6](1.400000+0.200000=1.600000)
V1[7]+V2[7]=V3[7](1.500000+0.100000=1.600000)
Tamaño vectores: 11
Numero de hilos: 4
Tiempo(seg.): 0.000002434
V1[0]=1.100000 V2[0]=1.100000 V3[0]=2.200000
V1[10]=2.100000 V2[10]=0.100000 V3[10]=2.200000
Vector resultado completo:
V1[0]+V2[0]=V3[0](1.100000+1.100000=2.200000)
V1[1]+V2[1]=V3[1](1.200000+1.000000=2.200000)
V1[2]+V2[2]=V3[2](1.300000+0.900000=2.200000)
V1[3]+V2[3]=V3[3](1.400000+0.800000=2.200000)
V1[4]+V2[4]=V3[4](1.500000+0.700000=2.200000)
V1[5]+V2[5]=V3[5](1.600000+0.600000=2.200000)
V1[6]+V2[6]=V3[6](1.700000+0.500000=2.200000)
V1[7]+V2[7]=V3[7](1.800000+0.400000=2.200000)
V1[8]+V2[8]=V3[8](1.900000+0.300000=2.200000)
V1[9]+V2[9]=V3[9](2.000000+0.200000=2.200000)
V1[10]+V2[10]=V3[10](2.100000+0.100000=2.200000)
```

8. Implementar un programa en C con OpenMP, a partir del código del Listado 1, que calcule en paralelo la suma de dos vectores usando las parallel y sections/section (se debe aprovechar el

paralelismo de datos usando estas directivas en lugar de la directiva for); es decir, hay que repartir el trabajo (tareas) entre varios threads usando sections/section. Se debe paralelizar también las tareas asociadas a la inicialización de los vectores. Para obtener este tiempo usar la función `omp_get_wtime()` en lugar de `clock_gettime()`. Tenga en cuenta lo mencionado anteriormente para la inicialización.

NOTAS: (1) el número de componentes N de los vectores debe ser un argumento de entrada al programa; (2) se deben inicializar los vectores antes del cálculo; (3) se debe asegurar que el programa calcula la suma correctamente imprimiendo todos los componentes del vector resultante, v3, para tamaños pequeños de los vectores (por ejemplo, N = 8); (4) se debe imprimir el tamaño de los vectores y el número de hilos; (5) sea cual sea el tamaño de los vectores el tiempo de ejecución del código paralelo que suma los vectores y, al menos, el primer y último componente de v1, v2 y v3 (esto último evita que las optimizaciones del compilador eliminen el código de la suma).

RESPUESTA: Captura que muestre el código fuente implementado `sp-OpenMP-sections.c`

```

37
38     #pragma omp sections
39     {
40         #pragma omp section
41         {
42             for (i = 0; i < mitad; i++) {
43                 v1[i] = N * 0.1 + i * 0.1;
44                 v2[i] = N * 0.1 - i * 0.1;
45             }
46         }
47
48         #pragma omp section
49         {
50             for (i = mitad; i < (int)N; i++) {
51                 v1[i] = N * 0.1 + i * 0.1;
52                 v2[i] = N * 0.1 - i * 0.1;
53             }
54         }
55     }
56
57     t1 = omp_get_wtime();
58
59
59
60     #pragma omp parallel
61     {
62         #pragma omp sections
63         {
64             #pragma omp section
65             {
66                 for (i = 0; i < mitad; i++) {
67                     v3[i] = v1[i] + v2[i];
68                 }
69             }
70
71             #pragma omp section
72             {
73                 for (i = mitad; i < (int)N; i++) {
74                     v3[i] = v1[i] + v2[i];
75                 }
76             }
77         }
78     }
79
80     t2 = omp_get_wtime();
81     tiempo = t2 - t1;
82

```

(RECUERDE ADJUNTAR CÓDIGO FUENTE AL .ZIP)

CAPTURAS DE PANTALLA (compilación y ejecución para N=8 y N=11):

```

david@DESKTOP-BQGQKB2:/mnt/c/Users/Daviton/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ S1)/Codigos$ export OMP_NUM_THREADS=2
./sumaVectorOMP2 8
./sumaVectorOMP2 11
Tamaño vectores: 8
Numero de hilos: 2
Tiempo(seg.): 0.000002584
V1[0]=0.800000 V2[0]=0.800000 V3[0]=1.600000
V1[7]=1.500000 V2[7]=0.100000 V3[7]=1.600000
Vector resultado completo:
V1[0]+V2[0]=V3[0](0.800000+0.800000=1.600000)
V1[1]+V2[1]=V3[1](0.900000+0.700000=1.600000)
V1[2]+V2[2]=V3[2](1.000000+0.600000=1.600000)
V1[3]+V2[3]=V3[3](1.100000+0.500000=1.600000)
V1[4]+V2[4]=V3[4](1.200000+0.400000=1.600000)
V1[5]+V2[5]=V3[5](1.300000+0.300000=1.600000)
V1[6]+V2[6]=V3[6](1.400000+0.200000=1.600000)
V1[7]+V2[7]=V3[7](1.500000+0.100000=1.600000)
Tamaño vectores: 11
Numero de hilos: 2
Tiempo(seg.): 0.000002087
V1[0]=1.100000 V2[0]=1.100000 V3[0]=2.200000
V1[10]=2.100000 V2[10]=0.100000 V3[10]=2.200000
Vector resultado completo:
V1[0]+V2[0]=V3[0](1.100000+1.100000=2.200000)
V1[1]+V2[1]=V3[1](1.200000+1.000000=2.200000)
V1[2]+V2[2]=V3[2](1.300000+0.900000=2.200000)
V1[3]+V2[3]=V3[3](1.400000+0.800000=2.200000)
V1[4]+V2[4]=V3[4](1.500000+0.700000=2.200000)
V1[5]+V2[5]=V3[5](1.600000+0.600000=2.200000)
V1[6]+V2[6]=V3[6](1.700000+0.500000=2.200000)
V1[7]+V2[7]=V3[7](1.800000+0.400000=2.200000)
V1[8]+V2[8]=V3[8](1.900000+0.300000=2.200000)
V1[9]+V2[9]=V3[9](2.000000+0.200000=2.200000)
V1[10]+V2[10]=V3[10](2.100000+0.100000=2.200000)

```

9. ¿Cuántos threads y cuántos cores como máximo podría utilizar la versión que ha implementado en el ejercicio 7? Razone su respuesta. ¿Cuántos threads y cuantos cores como máximo podría utilizar la versión que ha implementado en el ejercicio 8? Razone su respuesta. NOTA: Al contestar piense sólo en el **código, no piense** en el computador en el que lo va a ejecutar.

RESPUESTA:

En el ejercicio 7 el numero maximo de hilos y cores sería N, uno por cada iteración, el trabajo puede dividirse al estar en un bucle for.

Mientras que en el ejercicio 8 sería de 2 en ambos, ya que dividimos el trabajo en 2 secciones.

10. Rellenar una tabla como la Tabla 2 para atcgrid y otra para su PC con los tiempos de ejecución de los programas paralelos implementados en los ejercicios 7 y 8 y el programa secuencial del Listado 1. Generar los ejecutables usando -O2. Escribir un script para realizar las ejecuciones necesarias utilizando como base el script del seminario de BP0 (se deben imprimir en el script al menos las variables de entorno que ya se imprimen en el script de BP0). En la tabla debe aparecer el tiempo de ejecución del trozo de código que realiza la suma en paralelo (este es el tiempo que deben imprimir los programas). Ponga en la tabla el número de threads/cores que usan los códigos (use el máximo número de cores físicos del computador que como máximo puede aprovechar el código, no use un número de threads superior al número de cores físicos). Represente en una gráfica los tres tiempos. NOTA: Nunca ejecute código que imprima todos los componentes del resultado cuando este número sea elevado. Observar que el número de componentes en la tabla llega hasta 67108864.

RESPUESTA: Captura del script implementado sp-OpenMP-script10.sh

```

C SumaVectoresC-global.c M    C sumaVectorOMP.c U    C sumaVectorOMP2.c U    sp-OpenMP-script10.sh U X
AC > Practica 1 (+ S1) > Codigos > sp-OpenMP-script10.sh
1  #!/bin/bash
2
3  THREADS_FOR=6
4  THREADS_SECTIONS=2
5
6  echo
7  echo "=== VARIABLES DE ENTORNO OPENMP ==="
8  echo "OMP_NUM_THREADS=$OMP_NUM_THREADS"
9  echo "OMP_DYNAMIC=$OMP_DYNAMIC"
10 echo "OMP_SCHEDULE=$OMP_SCHEDULE"
11 echo
12
13 sizes=(16384 32768 65536 131072 262144 524288 1048576 2097152 4194304 8388608 16777216 33554432 67108864)
14
15 echo "===== "
16 echo "VERSION SECUENCIAL"
17 echo "===== "
18 for n in "${sizes[@]}"
19 do
20     echo "----- N=$n -----"
21     ./secuencial "$n"
22     echo
23 done
24
25 echo "===== "
26 echo "VERSION OPENMP FOR"
27 echo "Threads usados: $THREADS_FOR"
28 echo "===== "
29 export OMP_NUM_THREADS=$THREADS_FOR
30 for n in "${sizes[@]}"
31 do
32     echo "----- N=$n -----"
33     ./openmp_for "$n"
34     echo
35 done
36
37 echo "===== "
38 echo "VERSION OPENMP SECTIONS"
39 echo "Threads usados: $THREADS_SECTIONS"
40 echo "===== "
41 export OMP_NUM_THREADS=$THREADS_SECTIONS
42 for n in "${sizes[@]}"
43 do
44     echo "----- N=$n -----"
45     ./openmp_sections "$n"
46     echo
47 done

```

(RECUERDE ADJUNTAR LOS CÓDIGOS AL .ZIP)**CAPTURAS DE PANTALLA (mostrar la ejecución en atcgrid – envío(s) a la cola):**

```

david@DESKTOP-BQ6QK82: /mnt/c/Users/Davito/Desktop/Ingenieria-Informatica-2/AC/Practica 1 (+ SI)/Codigo$ ./sp-OpenMP-script10.sh

=== VARIABLES DE ENTORNO OPENMP ===
OMP_NUM_THREADS=2
OMP_DYNAMIC=
OMP_SCHEDULE=

=====
VERSION SECUENCIAL
=====
----- N=16384 -----
Tiempo(seg.):0.000115868 / Tamaño Vectores:16384 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[16383]+V2[16383]=V3[16383](0.401806+0.450813=0.852618) /
----- N=32768 -----
Tiempo(seg.):0.000150951 / Tamaño Vectores:32768 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[32767]+V2[32767]=V3[32767](0.550819+0.133142=0.683961) /
----- N=65536 -----
Tiempo(seg.):0.000391282 / Tamaño Vectores:65536 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[65535]+V2[65535]=V3[65535](0.823805+0.432539=1.256344) /
----- N=131072 -----
Tiempo(seg.):0.000808851 / Tamaño Vectores:131072 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[131071]+V2[131071]=V3[131071](0.269619+0.770286=1.039904) /
----- N=262144 -----
Tiempo(seg.):0.001831744 / Tamaño Vectores:262144 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[262143]+V2[262143]=V3[262143](0.760611+0.401589=1.162200) /
----- N=524288 -----
Tiempo(seg.):0.002870106 / Tamaño Vectores:524288 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[524287]+V2[524287]=V3[524287](0.140057+0.487439=0.627496) /
----- N=1048576 -----
Tiempo(seg.):0.005794809 / Tamaño Vectores:1048576 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[1048575]+V2[1048575]=V3[1048575](0.488793+0.952107=1.440900) /
----- N=2097152 -----
Tiempo(seg.):0.010574916 / Tamaño Vectores:2097152 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[2097151]+V2[2097151]=V3[2097151](0.545639+0.053318=0.598957) /
----- N=4194304 -----
Tiempo(seg.):0.030028283 / Tamaño Vectores:4194304 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[4194303]+V2[4194303]=V3[4194303](0.096832+0.943240=1.040071) /
----- N=8388608 -----
Tiempo(seg.):0.068619364 / Tamaño Vectores:8388608 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[8388607]+V2[8388607]=V3[8388607](0.949217+0.473084=1.422300) /
----- N=16777216 -----
Tiempo(seg.):0.139661551 / Tamaño Vectores:16777216 / V1[0]+V2[0]=V3[0](0.958116+0.094572=1.052688) // V1[16777215]+V2[16777215]=V3[16777215](0.653987+0.532771=1.186758) /

```

Tabla 2. Tiempos de ejecución de la versión secuencial de la suma de vectores y de las dos versiones paralelas. Sustituir en el encabezado de la tabla “¿?” por el número de threads utilizados, que debe coincidir con el número de cores físicos y cores lógicos utilizados.

Nº de Componentes	T. secuencial vect. Globales 1 thread=core	T. paralelo (versión for) 6 threads = cores lógicos = cores físicos	T. paralelo (versión sections) 2 threads = cores lógicos = cores físicos
16384	:0.000115868	0.000024923	0.000044657
32768	:0.000150951	0.000037919	0.000079076
65536	:0.000391282	0.000068564	0.000178042
131072	:0.000808851	0.000126169	0.000340971
262144	:0.001831744	0.000270764	0.000617297
524288	:0.002870106	0.000713082	0.002067278
1048576	:0.005794809	0.001532356	0.003611936
2097152	:0.010574916	0.007014255	0.006678824
4194304	:0.030028283	0.011364609	0.011632609
8388608	:0.068619364	0.014782943	0.026745230
16777216	:0.139661551	0.026816415	0.044758496
33554432	:0.283644413	0.054306917	0.091188111
67108864	:0.548024159	0.108137414	0.303555066

11. Rellenar una tabla como la Tabla 3 para atcgrid con el tiempo de ejecución, tiempo de CPU del usuario y tiempo CPU del sistema obtenidos con time para el ejecutable del ejercicio 7 y para el programa secuencial del Listado 1. Ponga en la tabla el número de threads (que debe coincidir con el número cores físicos y lógicos) que usan los códigos. Escribir un script para realizar las ejecuciones necesarias utilizando como base el script del seminario de BP0 (se deben imprimir en el script al menos las variables de entorno que ya se imprimen en el script de BP0) ¿El tiempo de CPU que se obtiene es mayor o igual que el tiempo real (*elapsed*)? Justifique la respuesta.

RESPUESTA: Captura del script implementado sp-OpenMP-script11.sh

```

sp-OpenMP-script11.sh U X
AC > Practica 1 (+ S1) > Codigos > sp-OpenMP-script11.sh
1  #!/bin/bash
2
3  THREADS_FOR=6
4  sizes=(8388608 16777216 33554432 67108864)
5
6  echo
7  echo "=== VARIABLES DE ENTORNO OPENMP ==="
8  echo "OMP_NUM_THREADS=$OMP_NUM_THREADS"
9  echo "OMP_DYNAMIC=$OMP_DYNAMIC"
10 echo "OMP_SCHEDULE=$OMP_SCHEDULE"
11 echo
12
13 echo "====="
14 echo "VERSION SECUENCIAL"
15 echo "====="
16 for n in "${sizes[@]}"
17 do
18     echo "----- N=$n -----"
19     /usr/bin/time -f "Elapsed=%e User=%U Sys=%S" ./secuencial "$n"
20     echo
21 done
22
23 echo "====="
24 echo "VERSION OPENMP FOR"
25 echo "Threads usados: $THREADS_FOR"
26 echo "====="
27 export OMP_NUM_THREADS=$THREADS_FOR
28 for n in "${sizes[@]}"
29 do
30     echo "----- N=$n -----"
31     /usr/bin/time -f "Elapsed=%e User=%U Sys=%S" ./openmp_for "$n"
32     echo
33 done

```

Tabla 3. Tiempos de ejecución de la versión secuencial de la suma de vectores y de las dos versiones paralelas. Sustituir en el encabezado de la tabla “¿?” por el número de threads utilizados.

Nº de Componentes	Tiempo secuencial vect. Globales 1 thread = 1 core lógico = 1 core físico			Tiempo paralelo/versión for 6 Threads = cores lógicos=cores físicos		
	<i>Elapsed</i>	<i>CPU-user</i>		<i>Elapsed</i>	<i>CPU-user</i>	<i>CPU-sys</i>
		<i>CPU-sys</i>			<i>sys</i>	
8388608	Elapsed=0.30	User=0.13	Sys=0.16	Elapsed=0.07	User=0.10	Sys=0.16
16777216	Elapsed=0.50	User=0.24	Sys=0.26	Elapsed=0.15	User=0.15	Sys=0.38
33554432	Elapsed=1.05	User=0.48	Sys=0.57	Elapsed=0.33	User=0.47	Sys=0.80
67108864	Elapsed=2.30	User=0.94	Sys=1.36	Elapsed=0.56	User=0.52	Sys=1.87

El tiempo en Secuencial debería de ser igual o aproximarse mucho, ya que solo hay un hilo funcionando, mientras que en la versión paralela, al haber varios hilos a la vez, la CPU debe trabajar más y el tiempo suele ser mayor que el real porque se acumulan.